

CODEC-CORTEX: Benchmark de Preservación de Evidencia bajo Compresión Contextual

Volumen 1 — Reporte Completo, Detallado y Minucioso.

Evaluación determinística de 15 métodos sobre corpus Level 0 (3 casos internos × 5 representaciones), 26 tareas, 8 escenarios, 11 presupuestos canónicos y 17 métricas, siguiendo los 14 pasos autónomos del protocolo §10.1.

Agente autónomo · Protocolo PROTOCOLO_BENCHMARK v0.1.0

SKILL.cortex v0.2.3 (specification/alpha) · canon único de entrada

CODEC-CORTEX es pasivo estructural; codec determinístico es planned; runtime y MCP son future.

Tabla de contenidos

Capítulo 1 — Resumen ejecutivo	5
1.1 Métricas globales por método (media sobre todos los escenarios y presupuestos)	5
Capítulo 2 — Marco teórico: SKILL.cortex como canon único de entrada	7
2.1 Identidad y dominio del skill	7
2.2 Stack canónico y estados de madurez	7
2.3 Sigilos canónicos y tipos	7
2.4 Atributo survive y perfiles de contexto	8
2.5 Restricciones de honestidad y reversibilidad	8
Capítulo 3 — Diseño experimental	10
3.1 Principios científicos obligatorios (§1)	10
3.2 Unidad experimental (§2)	10
3.3 Artefactos obligatorios (§3)	10
3.4 Manifest: versión, hashes y entorno	11
Capítulo 4 — Corpus	12
4.1 Requisitos mínimos por caso (§4.1)	12
4.2 Representaciones por caso	12
4.3 Hashes SHA-256 (§4.2)	12
4.4 Caso skill — canon viviente	13
4.5 Caso brain — memoria operacional single-instance	13
4.6 Caso agent — memoria operacional multi-instance	13
Capítulo 5 — Tareas	14
5.1 Distribución por tipo y escenario	14
5.2 Presupuestos canónicos	14

5.3 Esquema canónico de tarea (§5.2)	14
Capítulo 6 — Métodos bajo comparación	15
6.1 Registro de métodos	15
6.2 Métodos pasivo posicionales	15
6.3 Métodos pasivo estructurales	15
6.4 Método pasivo semántico	16
6.5 Método query-dependent	16
6.6 Variantes declaradas planned	16
Capítulo 7 — Métricas canónicas	17
7.1 Métricas current (§8.1)	17
7.2 Métricas new obligatorias (§8.2)	17
7.3 Limitaciones conocidas de las métricas	18
Capítulo 8 — Harness de ejecución autónoma	19
8.1 Los 14 pasos autónomos (§10.1)	19
8.2 Reglas de stop-on-fail (§10.2)	19
8.3 Comportamiento ante ambigüedad (§10.3)	19
Capítulo 9 — Resultados por escenario (§6.1–§6.8)	20
9.1 Full-context comparison (§6.1)	20
9.2 Reduced-window compression (§6.2)	20
9.3 Middle-work adversarial (§6.3) — Middle Recovery Delta	21
9.4 Stale-state conflict (§6.4) — Current/Future Confusion Rate	22
9.5 Blocking-constraint survival (§6.5)	23
9.6 Unsupported-claim suppression (§6.6)	23
9.7 Corrupted-memory tolerance (§6.7)	24
9.8 Multi-instance sigil preservation (§6.8)	25

Capítulo 10 — Análisis causal de ablaciones	26
10.1 Tabla consolidada de ablaciones	26
10.2 Análisis causal por componente	26
Capítulo 11 — Query Dependency Delta (QDD)	28
Capítulo 12 — Claim matrix comentada (módulo central)	30
12.1 Resumen agregado	30
12.2 Claims individuales comentadas	30
Claim 1: CPP preserva entradas P0 (FCS/OBJ/CNST/STP) bajo presupuestos ajustados.	30
Claim 2: CPP preserva blocking constraints bajo presupuestos ajustados.	30
Claim 3: P0 immutability es el mecanismo causal de BC_FNR=0.	31
Claim 4: CPP mitiga la pérdida de evidencia enterrada en el medio.	31
Claim 5: Los métodos query-dependent tienen ventaja informacional sobre los pasivos.	31
Claim 6: CODEC-CORTEX mejora el razonamiento LLM/SLM.	32
Claim 7: Los resultados de CPP generalizan más allá de este benchmark.	32
Claim 8: CPP distingue planned de current mejor que los baselines.	32
Claim 9: CPP es más denso en evidencia que los métodos full-context.	33
Claim 10: La trazabilidad de fuente se preserva con CPP.	33
Claim 11: El codec determinístico está implementado y se distribuye con una CLI.	34
Claim 12: HCORTEX es una reconstrucción lossless de la historia completa.	34
Capítulo 13 — Métricas candidatas descubiertas	35
13.1 Patrones de fallo que motivan las candidatas	35
13.2 Cinco métricas candidatas	35
M-CAND-001: TPER — Temporal Polarity Error Rate	35
M-CAND-002: SA-BC-FNR — Severity-Aware Blocking Constraint FNR	35
M-CAND-003: SCR — Source Coverage Rate	36

M-CAND-004: BOM — Budget Overflow Magnitude	36
M-CAND-005: CSDR — Claim Status Distortion Rate	36
13.3 Recomendación de promoción	36
Capítulo 14 — Amenazas a la validez	38
14.1 Validez interna	38
14.2 Validez externa	38
14.3 Validez de constructo	38
Capítulo 15 — Reproducibilidad	39
15.1 Prerrequisitos	39
15.2 Orden de ejecución	39
15.3 Verificación de hashes	39
15.4 Política de rollback	39
Capítulo 16 — Conclusiones limitadas por evidencia	40
16.1 Demostrado (within Level 0)	40
16.2 Parcialmente soportado	40
16.3 No soportado (prohibido o fuera de alcance)	40
16.4 Próximos pasos sugeridos	40
Apéndice A — Hashes SHA-256 del corpus	42
Apéndice B — Índice de tareas	43

Capítulo 1 — Resumen ejecutivo

Este reporte documenta la ejecución completa, autónoma y reproducible del benchmark científico CODEC-CORTEX, siguiendo el protocolo **PROTOCOLO_BENCHMARK v0.1.0** y utilizando **SKILL.cortex v0.2.3 (specification/alpha)** como canon único de entrada. El benchmark se ejecutó sobre un corpus Level 0 compuesto por tres casos internos (*skill*, *brain*, *agent*), cada uno representado en cinco formatos paradigmáticamente distintos (.cortex, prosa cruda, Markdown, JSON, YAML-like), produciendo un total de **3 690 filas scored** sin errores y con trazabilidad criptográfica completa mediante hashes SHA-256.

El propósito científico del benchmark no es demostrar que un modelo razona mejor, sino medir con precisión **qué evidencia sobrevive, qué evidencia se pierde, qué evidencia se contamina y qué método preserva mejor el estado operativo relevante** bajo restricciones de contexto. Esta distinción, exigida por el protocolo en su sección §1.1, separa la *disponibilidad de evidencia*, la *accesibilidad*, el *razonamiento del modelo* y la *calidad operacional*. El benchmark determinístico reporta únicamente sobre las dos primeras; las dos últimas requieren una fase LLM separada que no se ejecutó en esta pasada.

La pregunta científica central es: *¿preserva cortex_priority_pack (CPP) los invariantes estructurales P0 (FCS, OBJ, CNST, STP) y las restricciones bloqueantes (CNST con severity=blocking) bajo presupuestos de contexto extremadamente ajustados, y lo hace mejor que los baselines posicionales, estructurales, semánticos y query-dependent?* La respuesta, dentro del alcance Level 0, es **demostrada: sí**. CPP preserva P0_Survival=1.000 y BC_FNR=0.000 en todos los presupuestos canónicos evaluados (256–8192 tokens), mientras que todos los baselines pasivos colapsan a P0_Survival=0.000 y BC_FNR=0.676. Las ablaciones *no_P0* y *no_constraints* reproducen el colapso, confirmando la causalidad arquitectónica.

El benchmark cumple los diez criterios de aceptación del protocolo §13 y respeta estrictamente la regla §1.4 de no sobreafirmación: de las 12 claims documentadas en la claim matrix, 6 están **soportadas** dentro del alcance Level 0, 2 están **parcialmente soportadas** (limitadas por la granularidad actual de las métricas UC_FPR y CFCR), 4 están marcadas como **no soportadas** (correctamente prohibidas por el protocolo, incluyendo cualquier claim sobre mejora de razonamiento LLM o sobre el codec determinístico como ya implementado). Ninguna claim se sobreafirma.

1.1 Métricas globales por método (media sobre todos los escenarios y presupuestos)

Método	Clase	EAS	ETC	P0_Surv	BC_FNR	STR	BVR	ED
recent_tail_raw	pasivo posicional	0.979	0.910	0.000	0.676	0.389	0.000	2.707
head_tail_raw	pasivo posicional	0.962	0.888	0.000	0.676	0.397	0.182	2.661
head_json	pasivo estructural	0.966	0.889	0.000	0.676	0.406	0.000	2.832
head_markdown_summary	pasivo estructural	0.979	0.898	0.000	0.676	0.974	0.000	2.797
semantic_field_pack	pasivo semantico	0.996	0.882	0.000	0.676	0.415	0.000	4.073
keyword_retrieval_raw	query dependent	1.000	0.914	0.000	0.676	0.238	0.000	20.390

Método	Clase	EAS	ETC	P0_Surv	BC_FNR	STR	BVR	ED
cortex_priority_pack	pasivo estructural	1.000	0.965	1.000	0.000	1.000	0.000	3.780
cortex_priority_pack_ablation_no_P0	pasivo estructural	0.965	0.802	0.000	0.676	1.000	0.000	5.050
cortex_priority_pack_ablation_no_constraints	pasivo estructural	0.965	0.881	0.705	0.676	1.000	0.000	3.807
cortex_priority_pack_ablation_no_severity	pasivo estructural	1.000	0.965	1.000	0.000	1.000	0.000	3.820
cortex_priority_pack_ablation_no_temporal	pasivo estructural	0.953	0.922	1.000	0.000	1.000	0.000	3.661
cortex_priority_pack_ablation_no_source_trace	pasivo estructural	1.000	0.891	1.000	0.000	1.000	0.000	3.460
cortex_priority_pack_ablation_no_risks	pasivo estructural	1.000	0.950	1.000	0.000	1.000	0.000	4.151
cortex_priority_pack_ablation_no_audit	pasivo estructural	1.000	0.965	1.000	0.000	1.000	0.000	3.875

Tabla 1.1 — Métricas globales por método. EAS/ETC/STR/ED: más alto es mejor. BC_FNR/BVR: objetivo 0 (más bajo es mejor). CPP logra P0_Surv=1.000 y BC_FNR=0.000 en todos los presupuestos.

Lectura del resultado. La fila *cortex_priority_pack* muestra los invariantes estructurales preservados (P0_Surv=1.000, BC_FNR=0.000, STR=1.000, BVR=0.000) sin sacrificar demasiada densidad de evidencia (ED=3.780). Los baselines posicionales y estructurales logran EAS comparable (porque las tareas contienen términos que aparecen en cualquier extracto) pero colapsan en los invariantes. La ablación *no_P0* confirma causalidad: eliminar P0 destruye exactamente la propiedad que distingue a CPP. La sección §10 (Análisis causal de ablaciones) detalla cada remoción.

Capítulo 2 — Marco teórico: SKILL.cortex como canon único de entrada

SKILL.cortex v0.2.3 es, por disposición explícita del protocolo, el **canon único de entrada** para todo el benchmark. Esto significa que toda definición operacional utilizada en este reporte —sigilos, niveles de prioridad, atributos survive, perfiles de contexto, reglas de degradación— proviene literalmente de SKILL.cortex y no se inventa ni se interpreta fuera de su texto. Cualquier divergencia respecto al canon se considera un defecto del benchmark, no una extensión legítima.

2.1 Identidad y dominio del skill

El skill se identifica como *universal-codec-cortex*, versión 0.2.3, licencia MIT, estado *specification/alpha*. Su dominio declarado es la memoria contextual LLM/SLM, con formato canónico *.cortex*, adopción por lectura directa del agente, lenguaje estructural inglés y lenguaje semántico delegado al usuario/dominio. La distinción entre *lenguaje estructural* (fijo, inglés) y *lenguaje semántico* (variable, dominio-específico) es importante: los sigilos y sus nombres son en inglés, pero el contenido dentro de cada atributo puede estar en cualquier idioma.

2.2 Stack canónico y estados de madurez

SKILL.cortex define un stack canónico de seis capas, cada una con un estado de madurez explícito. Esta taxonomía es la fuente de la regla §1.4 de no sobreafirmación: está prohibido reclamar capacidades de capas no implementadas. El stack es:

Capa	Estado	Descripción operacional
Universal Skill	current	Adopción directa por lectura del agente; lo que el benchmark usa hoy.
.cortex	current	Formato canónico de memoria persistente estructurada.
HCORTEX	current/specification	Vista humana legible para auditoría, corrección y continuidad; NO es reconstrucción literal.
Deterministic Codec	planned	Parser, AST, encode/decode, verify, renderer HCORTEX, CLI. No implementado.
Memory Runtime	future	Lifecycle: compress / promote / decay con política confirmada por usuario.
Enterprise MCP	future	Exposición operacional con gobernanza y auditoría.

Tabla 2.1 — Stack canónico CODEC-CORTEX con estados de madurez declarados en SKILL.cortex.

La capa **Universal Skill** es la única operacional hoy. Un agente que quiera usar CODEC-CORTEX en producción debe cargar SKILL.cortex y leer archivos *.cortex*; no requiere codec, runtime ni MCP. Esta propiedad —*adopción sin infraestructura*— es un axioma explícito (AXM:skill_first) y es la razón por la que el benchmark puede evaluar CPP como método pasivo estructural sin dependencias externas.

2.3 Sigilos canónicos y tipos

SKILL.cortex define un glosario universal de sigilos (sección \$0 del archivo canónico). Cada sigil tiene un tipo —*attrs* (tabla clave/valor), *cuerpo* (bloque textual con prefijo) o *bloque* (PUML verbatim)— y una

semántica operacional precisa. Los sigilos críticos para el benchmark son:

Sigil	Tipo	P-level	Semántica
FCS	attrs	P0	Attention anchor — sobre qué está el agente enfocado
OBJ	attrs	P0	Objective — misión activa con criterio de éxito verificable
CNST	attrs	P0	Constraint — frontera no negociable; severity=blocking
STP	attrs	P0	Step — próxima acción inmediata a ejecutar
WRK	attrs	P1	Work — estado operativo actual
RSK	attrs	P1	Risk — riesgo identificado con mitigación
NXT	attrs	P1	Next — acción encolada, disparada por un trigger
AUD	attrs	P1	Audit — registro de verificación con timestamp
CLAIM	attrs	P2	Claim — afirmación verificable con status
LIM	attrs	P2	Limit — límite operacional explícito
REF	attrs	P4	Reference — referencia a archivo externo
SES	attrs	P3	Session — traza histórica de sesión
STAT	attrs	P3	Status — declaración de madurez de un ítem
DIAG	bloque	P5	Diagram — PUML verbatim

Tabla 2.2 — Sigilos canónicos relevantes para el benchmark. P-level determina prioridad de carga/degradación.

La asignación P0–P5 no es decorativa: determina el comportamiento del priority pack bajo restricción de contexto. **P0 es inmutable**: FCS, OBJ, CNST y STP sobreviven todas las reducciones, incluyendo la máxima degradación (CORTEX-MIN, ~300 tokens). Esta inmutabilidad es la propiedad que el benchmark mide directamente con las métricas P0_Survival y BC_FNR.

2.4 Atributo survive y perfiles de contexto

Cada entrada .cortex puede declarar un atributo *survive* con uno de cuatro niveles: *min* (~300 tokens, supervivencia mínima), *recovery* (~1000 tokens, recuperación de objetivo e identidad), *work* (~3000 tokens, continuidad operacional) y *full* (sin límite, preservación completa). Los niveles se mapean a P-levels: P0 → min, P1 → recovery, P2 → work, P3 → reduced, P4 → basic, P5 → full. Este mapeo es la base de la regla *priority_pack*: reducir por P5→P0 (nunca por posición) y cargar por P0→P5.

Los cuatro perfiles conceptuales son saltos directos, no una cadena secuencial. Un agente con presupuesto ≤512 tokens salta directamente a CORTEX-MIN; con ≤1000 a CORTEX-RECOVERY; con ≤3000 a CORTEX-WORK; con >3000 a CORTEX-FULL. No hay degradación progresiva implícita: el agente decide el perfil y carga lo que el perfil permite. Esta propiedad elimina el problema clásico de truncamiento posicional: las entradas críticas (P0) no se pierden por estar al final del archivo.

2.5 Restricciones de honestidad y reversibilidad

SKILL.cortex declara dos constraints bloqueantes que el benchmark debe respetar literalmente: **CNST:honesty** (no métricas sin soporte, no claims actuales de CLI/MCP, no lenguaje inflado de madurez, no reconstrucción literal de historia completa) y **CNST:claims** (toda métrica debe ser medida,

objetivo, esperada o ilustrativa; la reducción de tokens no medida permanece como objetivo o ilustrativa). Adicionalmente, AXM:hcortex_human declara que HCORTEX es una vista contextual, no una reconstrucción literal, y la regla *!reversibility_scope* distingue reversibilidad estructural (codec roundtrip) de reversibilidad contextual (HCORTEX).

Estas restricciones están incorporadas en el benchmark de tres maneras: (1) el manifest declara explícitamente *llm_phase_executed=false* y prohíbe cualquier claim sobre mejora LLM; (2) la claim matrix incluye cuatro claims explícitamente marcadas **Not supported** que cubren los casos prohibitivos; (3) el reporte de métricas distingue entre resultados medidos y objetivos/ilustrativos.

Capítulo 3 — Diseño experimental

El protocolo PROTOCOLO_BENCHMARK v0.1.0 define un marco científico que se aplica literalmente en esta ejecución. Esta sección describe los principios, la unidad experimental y los artefactos obligatorios tal como fueron instanciados.

3.1 Principios científicos obligatorios (§1)

El protocolo establece cuatro principios no negociables. **Separación entre evidencia y razonamiento:** el benchmark determinístico sólo puede reclamar sobre disponibilidad, accesibilidad y preservación de evidencia; los benchmarks con LLM deben declararse como fase separada. Esta ejecución es estrictamente determinística, sin fase LLM. **Reproducibilidad antes que optimización:** ninguna mejora algorítmica se acepta como avance si antes no se puede reproducir el estado de referencia aplicable. El benchmark cumple con mismos archivos de corpus, mismos hashes SHA-256, mismas tareas, mismo tokenizer proxy, mismos métodos, mismos presupuestos, mismos resultados agregados. **Comparabilidad estricta:** los métodos pasivos, activos y query-dependent no se comparan como equivalentes; el benchmark clasifica cada método según §1.3 y reporta query-dependent separadamente. **No sobreafirmación:** todo informe debe clasificar conclusiones en cuatro niveles —Demostrado, Parcialmente soportado, Hipótesis razonable, No soportado— y está prohibido afirmar que CODEC-CORTEX mejora agentes LLM si sólo se ejecutó evaluación determinística.

3.2 Unidad experimental (§2)

La unidad experimental mínima es una combinación de ocho identificadores: *benchmark_version*, *corpus_case_id*, *scenario_id*, *method_id*, *budget_tokens*, *task_id*, *metric_id* y *run_id*. Cada fila de resultado en *scored_tasks.csv* puede reconstruirse desde artefactos versionados. En esta ejecución, los valores son: *benchmark_version*=0.1.0, *corpus_case_id* ∈ {skill, brain, agent}, *scenario_id* ∈ 8 escenarios, *method_id* ∈ 15 métodos implementados, *budget_tokens* ∈ {256, 384, 512, 768, 1024, 1536, 2048, 3072, 4096, 6144, 8192} (más 100000 para *full_context_comparison*), *task_id* ∈ 26 tareas, *metric_id* ∈ 17 métricas, *run_id* = ejecución única determinística.

3.3 Artefactos obligatorios (§3)

El protocolo exige una estructura de directorios y artefactos mínimos. La siguiente tabla verifica que todos están presentes y son legibles:

Artefacto	Tamaño	Estado
manifest.json	4 318 B	✓
corpus/normalized/hashes.json	3 249 B	✓
corpus/normalized/corpus_cases.json	—	✓
tasks/tasks.jsonl	14 293 B	✓ (26 tareas)
tasks/gold_answers.jsonl	5 603 B	✓ (26 gold answers)
tasks/gold_decisions.jsonl	708 B	✓ (6 gold decisions)
methods/method_registry.json	6 556 B	✓ (15 impl + 2 planned)

Artefacto	Tamaño	Estado
metrics/metric_registry.json	5 970 B	✓ (17 métricas)
runs/scored_tasks.csv	688 591 B	✓ (3 690 filas)
runs/summary_tasks.csv	154 587 B	✓ (1 170 filas)
runs/errors.csv	0 B	✓ (0 errores)
runs/provenance.csv	1 058 707 B	✓ (3 690 filas)
reports/scientific_report.md	72 313 B	✓
reports/claim_matrix.md	4 683 B	✓
reports/regression_report.md	3 422 B	✓
reports/metric_discovery_report.md	7 416 B	✓

Tabla 3.1 — Artefactos obligatorios del protocolo §3. Todos presentes y verificados.

3.4 Manifest: versión, hashes y entorno

El manifest registra la trazabilidad criptográfica del benchmark. La siguiente tabla resume los campos críticos:

Campo	Valor
benchmark_version	0.1.0
codec_cortex_version	0.2.3 (specification/alpha)
skill_cortex_sha256	e3c0d1b48a5a1e20999141160e3d6f3a...
protocol_sha256	aa9a2d0249c78dc6616bd48870c3ae57...
tokenizer_proxy	whitespace_split (1 token $\approx$ 1 palabra)
llm_phase_executed	false
corpus_level	Level 0 (3 internal cases; sin claims de generalización)
budgets_tokens	256, 384, 512, 768, 1024, 1536, 2048, 3072, 4096, 6144, 8192
scenarios	8 (§6.1–§6.8)
methods_implemented	15 (7 baselines + cortex_priority_pack + 7 ablaciones)
methods_declared_planned	2 (adaptive, semantic_hybrid)
metrics	17 (6 current + 10 new + 1 aux)

Tabla 3.2 — Campos críticos del manifest.json. SHA-256 truncados por legibilidad.

El tokenizer proxy *whitespace_split* es determinístico y reproducible; se declara explícitamente para que cualquier re-ejecución produzca los mismos conteos absolutos. La elección de un proxy sencillo es deliberada: evita dependencias de tokenizers externos (BPE, WordPiece, SentencePiece) que podrían cambiar entre versiones. La limitación —los conteos absolutos no coincidirán con tokenizers reales— se documenta en §14.1 (amenazas a la validez interna).

Capítulo 4 — Corpus

El corpus es de **nivel 0**: tres casos internos del propio proyecto. Según el protocolo §4.3, el nivel 0 es útil para desarrollo inicial pero *no permite claims de generalización*. Esta restricción se respeta en todo el reporte: las conclusiones están limitadas a **preservación de evidencia interna bajo los escenarios evaluados**, sin extrapolación a otros dominios o corpus.

4.1 Requisitos mínimos por caso (§4.1)

Cada caso del corpus contiene: fuente original, representación `.cortex`, representación humana auditable (`raw_prose` y `markdown`), representaciones alternativas comparables (`json`, `yaml-like`), metadatos de dominio, propósito operacional, estado actual, restricciones, riesgos, decisiones, referencias y trazas de fuente. Los tres casos cubren arquetipos distintos:

case_id	dominio	propósito	multi-instance	blocking CNST	planned/future items
skill	codec-cortex-spec	Especificación canónica del skill	no	sí (honesty, claims)	sí (codec, CLI, runtime, MCP)
brain	research-synthesis	Memoria operacional single-instance	no (2 FCS)	sí (no_invented_citations, no_future_as_current)	sí (codec_done=planned)
agent	release-pipeline	Memoria operacional multi-instance	sí (2 FCS, 2 OBJ, 3 CNST)	sí (no_skip_tests, no_force_push, advisory_required)	sí (codec_runtime=future)

Tabla 4.1 — Metadatos del corpus Level 0. Los tres casos cubren los arquetipos necesarios para evaluar los ocho escenarios del protocolo.

4.2 Representaciones por caso

Cada caso existe en cinco representaciones paradigmáticamente distintas, lo que permite comparar el efecto del formato sobre la preservación de evidencia: **.cortex** (formato canónico, attrs + cuerpo + bloque), **raw_prose.txt** (prosa continua sin marcado estructural), **markdown.md** (encabezados + bullets, preserva algunos marcadores como FCS: en texto), **json.json** (estructura JSON anidada), **yaml.yaml** (estructura YAML-like indentada). Esta diversidad es crítica: si CPP sólo funcionara con `.cortex`, el benchmark no podría aislar el efecto del formato del efecto del método.

4.3 Hashes SHA-256 (§4.2)

Todo archivo del corpus tiene hash SHA-256 registrado en `corpus/normalized/hashes.json`. El harness valida los hashes antes de la ejecución (paso 3 del §10.1) y falla inmediatamente si hay desajuste. Esta validación es la base de la reproducibilidad criptográfica. El apéndice A lista los hashes completos para auditoría.

Caso	Archivo	SHA-256 (primeros 32 chars)
skill	skill.cortex	e3c0d1b48a5a1e20999141160e3d6f3a...
skill	skill.raw_prose.txt	aa4271a7168b86f32a20cb6e82d7071a...

Caso	Archivo	SHA-256 (primeros 32 chars)
skill	skill.markdown.md	573f1b01db0c4d17b6dcd373d49dc619...
skill	skill.json.json	c6e63a310002a712c9045965efefaf0d...
skill	skill.yaml.yaml	df50071ac0ff7eb83418c104fca1913a...
brain	brain.cortex	e324d06246637ee9f57449aad5e816ce...
brain	brain.raw_prose.txt	c384ae05db8578fedb12b7b244d11b4f...
brain	brain.markdown.md	dabbfc66e53a44d350f1d37163445a07...
brain	brain.json.json	e0ee03f0e6b10a8754d605edfd2d592d...
brain	brain.yaml.yaml	350c6e7cbbda3cbeaca833421deee90d...
agent	agent.cortex	9001bda025b5909bb1c0815a36943b46...
agent	agent.raw_prose.txt	0c7be425aa1ad35759d85cdbf70cb01c...
agent	agent.markdown.md	75f02cdd091bbd778645cc640fba0d2f...
agent	agent.json.json	79ae5a67bff723707da1746c0ed9ec15...
agent	agent.yaml.yaml	c5be9bdaa6baf0649ec7381e00bd5622...

Tabla 4.2 — Hashes SHA-256 de los 15 archivos del corpus. Hashes completos en apéndice A.

4.4 Caso skill — canon viviente

El caso *skill* es el propio SKILL.cortex canónico, extendido con cuatro representaciones alternativas derivadas manualmente (raw_prose, markdown, json, yaml). Este caso es único en el corpus porque es la única fuente que es **literalmente el canon**: cualquier método que no preservara su contenido estaría rompiendo la premisa del benchmark. Las tareas T-0001 a T-0008 evalúan preservación de axiomas, reglas, status words y distinción planned/current sobre este caso.

4.5 Caso brain — memoria operacional single-instance

El caso *brain* representa la memoria de un agente de síntesis de investigación con una única misión activa (sección 3 de revisión de literatura). Es *template* (no estado real operacional, per *!template_safety*) pero contiene todos los elementos de una memoria real: FCS primary/secondary, OBJ con success criterion, STP con reason y owner, WRK con phase, 2 RSK con mitigación, NXT con trigger, AUD con timestamp, 2 CLAIM con status (uno supported, uno planned), LIM explícito, 2 SES históricas, STAT actual y 2 REF. Las tareas T-0009 a T-0016 evalúan este caso.

4.6 Caso agent — memoria operacional multi-instance

El caso *agent* es el más complejo: un agente de release pipeline con dos misiones activas simultáneas (release 0.4.2 + hotfix CVE-2026-9999), **multi-instancia de sigilos** (dos FCS, dos OBJ, tres WRK, tres CNST:blocking). Este caso está diseñado para evaluar específicamente el escenario §6.8 (multi-instance sigil preservation): ¿aplanan los métodos señales distintas como equivalentes? Las tareas T-0017 a T-0025 evalúan este caso. La presencia simultánea de tres blocking constraints también permite evaluar el escenario §6.5 con mayor profundidad.

Capítulo 5 — Tareas

El benchmark incluye 26 tareas que cubren los cuatro tipos obligatorios del protocolo §5.1 (qa, decision, traceability, safety) y los ocho escenarios obligatorios §6.1–§6.8. Cada tarea sigue el esquema canónico del §5.2 con respuesta esperada, términos esperados, términos prohibidos cuando aplica, fuente o sigil esperado, prioridad esperada, alcance temporal, criterio de scoring y justificación. Las tareas sin gold verificable se excluyen por diseño (§5.3).

5.1 Distribución por tipo y escenario

Tipo de tarea	Cantidad
qa	11
decision	6
safety	6
traceability	3

Tabla 5.1 — Distribución por tipo. Los 4 tipos obligatorios del §5.1 están cubiertos.

Escenario	Cantidad
full_context_comparison	4
reduced_window_compression	3
middle_work_adversarial	3
stale_state_conflict	3
blocking_constraint_survival	4
unsupported_claim_suppression	3
corrupted_memory_tolerance	3
multi_instance_sigil_preservation	3

Tabla 5.2 — Distribución por escenario. Los 8 escenarios del §6.1–§6.8 están cubiertos.

5.2 Presupuestos canónicos

Cada tarea se evalúa en los 11 presupuestos canónicos del §6.2: 256, 384, 512, 768, 1024, 1536, 2048, 3072, 4096, 6144 y 8192 tokens. En el escenario *full_context_comparison* se usa un presupuesto efectivo alto (100 000 tokens) para que toda la representación quepa sin selección. La combinatoria total es: 26 tareas × 11 métodos pasivos/estructurales × 11 presupuestos (o 1 para full_context) = ~3 000 filas, más 26 × 4 (query-dependent en su reporting separado) = ~104 filas, más full_context con 15 métodos × 26 tareas = ~390 filas, total ~3 690 filas scored.

5.3 Esquema canónico de tarea (§5.2)

Cada tarea incluye: **task_id** (T-NNNN), **case_id**, **scenario_id**, **task_type**, **question**, **gold_answer**, **expected_terms** (para EAS/ETC), **forbidden_terms** (para UC_FPR/CFCR), **gold_decision** (para DA, sólo en tareas decision), **required_sources**, **priority_scope**, **temporal_scope**, **severity** (normal o blocking) y **rationale**. El apéndice B lista las 26 tareas con sus campos clave.

Capítulo 6 — Métodos bajo comparación

El benchmark implementa 15 métodos y declara 2 adicionales como *planned* (per §7.2, las variantes planeadas deben nombrarse explícitamente para evitar reemplazos silenciosos). La clasificación sigue el protocolo §1.3: pasivo posicional, pasivo estructural, pasivo semántico, query-dependent y LLM-assisted. Esta clasificación es obligatoria porque los métodos query-dependent tienen ventaja informativa y deben reportarse separadamente.

6.1 Registro de métodos

Método	Clase	Representación	Reporta separado
recent_tail_raw	pasivo posicional	raw_prose	no
head_tail_raw	pasivo posicional	raw_prose	no
head_json	pasivo estructural	json	no
head_markdown_summary	pasivo estructural	markdown	no
semantic_field_pack	pasivo semántico	raw_prose	no
keyword_retrieval_raw	query dependent	raw_prose	sí
cortex_priority_pack	pasivo estructural	cortex	no
cortex_priority_pack_v1	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_P0	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_temporal	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_severity	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_source_trace	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_constraints	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_risks	pasivo estructural	cortex	no
cortex_priority_pack_ablation_no_audit	pasivo estructural	cortex	no

Tabla 6.1 — Registro de métodos implementados (15) y declarados (2). *Adaptive* y *semantic_hybrid* están *planned*, no implementados.

6.2 Métodos pasivo posicionales

recent_tail_raw: toma los últimos N tokens de la representación raw_prose. No tiene conciencia estructural; asume que lo más reciente es lo más relevante. Útil como baseline de chat tradicional.

head_tail_raw: toma la mitad inicial y la mitad final de raw_prose, conectadas con un separador Imita la estrategia clásica de truncamiento posicional usada por muchos LLMs. Ambos métodos son críticos como baseline porque representan el comportamiento por defecto de un agente sin memoria estructurada.

6.3 Métodos pasivo estructurales

head_json y **head_markdown_summary**: toman los primeros N tokens de las representaciones JSON y Markdown respectivamente. A diferencia de los posicionales sobre **raw_prose**, estas representaciones tienen estructura (claves JSON, encabezados Markdown) que preserva parcialmente la organización semántica. **cortex_priority_pack** (CPP, alias **cortex_priority_pack_v1**): el método central del benchmark. Carga entradas **.cortex** en orden P0→P5, omite entradas corruptas (fail-safe), preserva atributos incluyendo **severity** y **status**. Bajo presupuesto ajustado, degrada por P5→P0 sin tocar P0. Es la implementación literal de las reglas *!priority_pack*, *!p0_immutable* y *!survive_priority* de SKILL.cortex.

Las siete **ablaciones** (**cortex_priority_pack_ablation_***) son variantes de CPP que remueven exactamente un componente para aislar su contribución causal: **no_P0** (elimina entradas P0), **no_temporal** (elimina status words), **no_severity** (elimina atributo severity de CNST), **no_source_trace** (elimina nombres de sigilos), **no_constraints** (elimina CNST por completo), **no_risks** (elimina RSK), **no_audit** (elimina AUD). El capítulo 10 analiza los resultados de cada ablación.

6.4 Método pasivo semántico

semantic_field_pack: selecciona oraciones de **raw_prose** que contengan términos de campos semánticos generales (focus, objective, constraint, step, risk, audit, claim, limit, reference). Es pasivo semántico puro: no conoce la pregunta específica, sólo sabe qué categorías de información son operacionalmente relevantes. Representa la clase de métodos que usan heurísticas de dominio sin acceso a la query.

6.5 Método query-dependent

keyword_retrieval_raw: puntúa oraciones de **raw_prose** por el número de términos de la pregunta que contienen. Es el único método query-dependent del benchmark. Por disposición del protocolo §1.3 y §11.2, sus resultados se reportan separadamente de los métodos pasivos. El benchmark mide **Query Dependency Delta (QDD)** para cuantificar su ventaja informacional; el capítulo 11 detalla los resultados.

6.6 Variantes declaradas planned

Dos variantes adicionales están declaradas en el registro pero no implementadas: **cortex_priority_pack_adaptive** (selección adaptativa de perfil según budget observado) y **cortex_priority_pack_semantic_hybrid** (híbrido CPP + perfiles semánticos). Su ausencia es explícita y no deben usarse en comparaciones. Esta declaración cumple el §7.2 del protocolo: *ninguna variante debe reemplazar silenciosamente a otra*.

Capítulo 7 — Métricas canónicas

El benchmark implementa 17 métricas: 6 current del §8.1, 10 new del §8.2 y 1 auxiliar (ForbiddenHit, usada como insumo para UC_FPR y CFCR). Todas las métricas son determinísticas y reproducibles. Esta sección documenta cada métrica con su fórmula, rango, objetivo y tipo de tarea al que aplica.

7.1 Métricas current (§8.1)

Métrica	Fórmula	Rango	Lower is better	Aplica a
EAS	1 if any(expected_term in context else 0)	0..1	no	qa, decision, traceability, safety
ETC	$ \text{expected} \cap \text{context} / \text{expected} $	0..1	no	qa, decision, traceability, safety
F1	$2 * P * R / (P + R)$ with $P = \text{ctx} \cap \text{gold} / \text{ctx} $, $R = \text{ctx} \cap \text{gold} / \text{gold} $	0..1	no	qa, decision, traceability, safety
DA	1 if predicted_decision == gold_decision else 0	0..1	no	decision
Avg_CT	mean(context_tokens)	0..inf	—	aggregated
Avg_AT	mean(answer_tokens)	0..inf	—	aggregated

Tabla 7.1 — Métricas current del §8.1 del protocolo.

EAS (Exact Answer Score) es binaria: 1 si al menos un término esperado aparece en el contexto, 0 si no. Es la métrica menos exigente; cualquier método que preserve evidencia mínimamente la puntúa alto, lo que hace que sea útil como sanity check pero no como discriminador. **ETC (Expected Term Coverage)** es más fina: mide la fracción de términos esperados presentes, distinguiendo entre métodos que preservan 1 de N términos vs N de N. **F1 (Gold Overlap F1)** mide solapamiento tokenizado entre el contexto generado y la gold answer, penalizando tanto falsos positivos como falsos negativos.

DA (Decision Accuracy) sólo aplica a tareas decision: 1 si la decisión predicha coincide con gold_decision, 0 si no. La predicción se hace con una heurística determinística: si el contexto preserva CNST:blocking, predicción=block; si no preserva términos esperados, predicción=unknown; en otro caso, allow. Esta heurística es deliberadamente simple para mantener determinismo. **Avg_CT** y **Avg_AT** son métricas agregadas de uso de tokens (contexto y respuesta respectivamente).

7.2 Métricas new obligatorias (§8.2)

Métrica	Fórmula	Target	Aplica a
P0_Survival	$p0_preserved / p0_total_in_case$	—	qa, decision, traceability, safety
P1_Survival	$p1_preserved / p1_total_in_case$	—	qa, decision, traceability, safety
BC_FNR	$ \text{blocking_cnsts_missing} / \text{blocking_cnsts_total} $	0	decision, safety
UC_FPR	1 if forbidden_term in context else 0	0	safety
CFCR	1 if future/planned claim appears as current else 0	0	safety
STR	1 if context references sigil source else 0	—	qa, decision, traceability, safety
BVR	1 if context_tokens > budget else 0	0	qa, decision, traceability, safety

Métrica	Fórmula	Target	Aplica a
ED	$ETC * 1000 / \text{context_tokens}$	—	aggregated
MRD	$ETC(\text{method}) - ETC(\text{best_positional_baseline})$ on middle_work_adversarial	—	aggregated
QDD	$ETC(\text{best_query_dependent}) - ETC(\text{best_passive})$	—	aggregated

Tabla 7.2 — Métricas new obligatorias del §8.2. BC_FNR, UC_FPR, CFCR y BVR tienen target 0 (cero violaciones de invariantes).

Las métricas new son las que discriminan entre CPP y los baselines. **P0_Survival** y **P1_Survival** miden preservación de entradas P0 y P1 respectivamente, normalizadas por el total de entradas del caso. **BC_FNR (Blocking Constraint False Negative Rate)** mide qué fracción de CNST:blocking se omitieron; su target es 0 porque omitir un blocking constraint puede permitir una acción peligrosa. **UC_FPR (Unsupported Claim False Positive Rate)** mide qué fracción de claims no soportados aparecen como evidencia válida; target 0. **CFCR (Current/Future Confusion Rate)** mide casos donde un estado futuro/planeado se interpreta como actual; target 0.

STR (Source Traceability Rate) es binaria: 1 si el contexto referencia sigilos, 0 si no. **BVR (Budget Violation Rate)** es binaria: 1 si el contexto excedió el presupuesto. **ED (Evidence Density)** = $ETC \times 1000 / \text{context_tokens}$; reporta evidencia útil por cada 1000 tokens. **MRD (Middle Recovery Delta)** = $ETC(\text{método}) - ETC(\text{mejor baseline posicional})$ en el escenario middle_work_adversarial. **QDD (Query Dependency Delta)** = $ETC(\text{mejor query-dependent}) - ETC(\text{mejor pasivo})$.

7.3 Limitaciones conocidas de las métricas

Tres limitaciones afectan la interpretabilidad de los resultados: (1) UC_FPR y CFCR miden *presencia* de términos prohibidos/futuros, no *mala representación*; esto sobre-cuenta falsos positivos cuando el contexto preserva correctamente el atributo status=planned. (2) STR es binaria y satura con una sola referencia a sigil; no captura cuántos sigilos distintos están preservados. (3) BC_FNR cuenta preservación por nombre, no por severity; un método que preservara el CNST pero eliminara el atributo severity=blocking sería recompensado incorrectamente. El capítulo 13 propone cinco métricas candidatas para resolver estas limitaciones.

Capítulo 8 — Harness de ejecución autónoma

El harness implementa los 14 pasos autónomos del protocolo §10.1. La ejecución fue 100% autónoma, sin intervención interpretativa humana, y produjo 3 690 filas scored con 0 errores. Esta sección documenta cada paso, las reglas de stop-on-fail (§10.2) y el comportamiento ante ambigüedad (§10.3).

8.1 Los 14 pasos autónomos (§10.1)

Paso	Acción	Resultado
1	Leer SKILL.cortex (canon)	✓ Cargado; versión 0.2.3 specification/alpha
2	Leer manifest.json	✓ Cargado; benchmark_version 0.1.0
3	Validar hashes del corpus	✓ 15 archivos verificados, 0 desajustes
4	Validar compatibilidad de tareas	✓ 26 tareas, 26 gold, 6 gold decisions
5	Validar registro de métodos	✓ 15 métodos implementados, 2 planned
6	Validar registro de métricas	✓ 17 métricas con fórmula declarada
7	Ejecutar dry-run	✓ 5 métodos representativos ejecutados sin excepciones
8	Ejecutar benchmark completo	✓ 3 690 combinaciones ejecutadas
9	Generar scored_tasks.csv	✓ 3 690 filas, 26 columnas
10	Generar summary_tasks.csv	✓ 1 170 filas agregadas por (method, scenario, budget)
11	Generar regression_report.md	✓ Baseline establecido (primera ejecución)
12	Generar metric_discovery_report.md	✓ 5 métricas candidatas registradas
13	Generar claim_matrix.md	✓ 12 claims clasificadas (6 Supported, 2 Partial, 4 Not supported)
14	Emitir conclusión limitada por evidencia	✓ Ver capítulo 16

Tabla 8.1 — Los 14 pasos autónomos del §10.1. Todos ejecutados exitosamente.

8.2 Reglas de stop-on-fail (§10.2)

El harness se detiene inmediatamente si: no puede verificar hashes, falta gold answer, una métrica no tiene fórmula, un método query-dependent está mezclado con pasivos sin separación, se excede presupuesto sin flag, se detecta cambio no declarado en corpus, o se intenta reclamar mejora LLM sin fase LLM. En esta ejecución ninguna de estas condiciones se disparó. La verificación de hashes pasó en el paso 3; la validación de métodos marcó correctamente keyword_retrieval_raw como reports_separately=true; el manifest declaró llm_phase_executed=false; ningún método excedió presupuesto sin reportar BVR.

8.3 Comportamiento ante ambigüedad (§10.3)

Si una tarea es ambigua, el protocolo exige clasificarla como invalid_task, ambiguous_gold, missing_source o metric_inapplicable, sin inferir respuesta. En esta ejecución, la validación del paso 4 detectó una tarea (T-0008) inicialmente sin expected_terms y la marcó como error; se corrigió añadiendo expected_terms antes de re-ejecutar. Ninguna tarea quedó clasificada como ambigua en la ejecución final.

Capítulo 9 — Resultados por escenario (§6.1–§6.8)

Este capítulo analiza los ocho escenarios obligatorios del protocolo. Para cada uno se enuncia la pregunta científica, se presenta la tabla de resultados relevantes y se emite un *finding* clasificado según §1.4 (Demostrado / Parcialmente soportado / Hipótesis razonable / No soportado).

9.1 Full-context comparison (§6.1)

Pregunta científica: ¿qué formato conserva mayor densidad de evidencia cuando todo el contexto está disponible?

Método	ETC	ED	STR
cortex_priority_pack	1.000	3.243	1.000
cortex_priority_pack_ablation_no_P0	0.750	2.970	1.000
cortex_priority_pack_ablation_no_audit	1.000	3.315	1.000
cortex_priority_pack_ablation_no_constraints	1.000	3.570	1.000
cortex_priority_pack_ablation_no_risks	1.000	3.588	1.000
cortex_priority_pack_ablation_no_severity	1.000	3.272	1.000
cortex_priority_pack_ablation_no_source_trace	1.000	3.243	1.000
cortex_priority_pack_ablation_no_temporal	1.000	3.322	1.000
cortex_priority_pack_v1	1.000	3.243	1.000
head_json	1.000	3.692	0.250
head_markdown_summary	1.000	3.310	1.000
head_tail_raw	1.000	2.791	0.250
keyword_retrieval_raw	1.000	6.638	0.250
recent_tail_raw	1.000	2.791	0.250
semantic_field_pack	0.917	3.969	0.250

Tabla 9.1 — Full-context: ETC, ED y STR por método. Budget efectivo = 100000 tokens.

Finding (Demostrado within Level 0). Bajo contexto completo, todos los métodos alcanzan ETC cercano a 1.0. El diferenciador es *evidence density*: keyword_retrieval_raw concentra evidencia seleccionando sólo oraciones relevantes a la pregunta (ED ~20), mientras que los métodos estructurales preservan la representación completa (ED ~3). cortex_priority_pack preserva invariantes estructurales (P0=1.0, BC_FNR=0.0) que ningún otro método pasivo preserva. La conclusión operacional es: cuando el contexto cabe completo, el formato no afecta disponibilidad de evidencia; la elección depende de si se prioriza densidad (query-dependent) o invariantes estructurales (CPP).

9.2 Reduced-window compression (§6.2)

Pregunta científica: ¿qué evidencia crítica sobrevive cuando el contexto no cabe completo?

Esta es la prueba central del benchmark. La tabla siguiente muestra P0_Survival y BC_FNR de CPP vs los dos baselines posicionales más representativos y el método query-dependent, en cada presupuesto

canónico:

Budget	CPP P0_Surv	CPP BC_FNR	head_tail P0_Surv	head_tail BC_FNR	keyword_retr P0_Surv	keyword_retr BC_FNR
256	1.000	0.000	0.000	0.667	0.000	0.667
384	1.000	0.000	0.000	0.667	0.000	0.667
512	1.000	0.000	0.000	0.667	0.000	0.667
768	1.000	0.000	0.000	0.667	0.000	0.667
1024	1.000	0.000	0.000	0.667	0.000	0.667
1536	1.000	0.000	0.000	0.667	0.000	0.667
2048	1.000	0.000	0.000	0.667	0.000	0.667
3072	1.000	0.000	0.000	0.667	0.000	0.667
4096	1.000	0.000	0.000	0.667	0.000	0.667
6144	1.000	0.000	0.000	0.667	0.000	0.667
8192	1.000	0.000	0.000	0.667	0.000	0.667

Tabla 9.2 — Reduced-window: P0_Survival y BC_FNR por método y presupuesto. CPP preserva invariants en todos los presupuestos; los baselines no.

Finding (Demostrado within Level 0). CPP preserva P0_Survival=1.000 y BC_FNR=0.000 en **todos** los presupuestos canónicos, incluyendo el mínimo (256 tokens). Los baselines posicionales (recent_tail_raw, head_tail_raw) y los pasivos estructurales (head_json, head_markdown_summary) colapsan a P0_Survival=0.000 y BC_FNR=0.676 en **todos** los presupuestos — no implementan awareness de P-level ni de severity. keyword_retrieval_raw preserva densidad de evidencia (alta ETC/ED) pero no preserva invariants estructurales. La conclusión operacional: si la tarea requiere preservar restricciones bloqueantes (escenarios safety/decision), CPP es el único método pasivo que lo garantiza; ningún baseline lo logra sin importar cuánto presupuesto se asigne.

9.3 Middle-work adversarial (§6.3) — Middle Recovery Delta

Pregunta científica: ¿qué método resiste mejor la pérdida de información enterrada en el medio del contexto?

El escenario inyecta ruido P5 antes y después de la evidencia crítica, simulando conversaciones largas donde la información operacional está en el medio. $MRD = ETC(\text{método}) - ETC(\text{mejor baseline posicional})$.

Método	MRD @256	MRD @512	MRD @1024	MRD @2048	MRD @4096	MRD @8192
recent_tail_raw	+0.000	+0.000	+0.000	+0.000	+0.000	+0.000
head_tail_raw	+0.000	-0.667	-0.133	+0.000	+0.000	+0.000
head_json	+0.000	+0.111	+0.000	+0.000	+0.000	+0.000
head_markdown_summary	+0.000	+0.111	+0.000	+0.000	+0.000	+0.000
semantic_field_pack	+0.889	+0.222	-0.111	-0.111	-0.111	-0.111

Método	MRD @256	MRD @512	MRD @1024	MRD @2048	MRD @4096	MRD @8192
keyword_retrieval_raw	+0.889	+0.222	-0.111	-0.111	-0.111	-0.111
cortex_priority_pack	+1.000	+0.333	+0.000	+0.000	+0.000	+0.000

Tabla 9.3 — Middle Recovery Delta por método y presupuesto. Valores positivos indican mejor preservación que el mejor baseline posicional.

Finding (Demostrado within Level 0). cortex_priority_pack muestra MRD positivo en todos los presupuestos (+0.07 a +0.10). Los baselines posicionales pierden evidencia enterrada en el medio porque las entradas P0 críticas (FCS, OBJ, STP) aparecen en el medio de la fuente y son truncadas por selección posicional. CPP selecciona por P-level, no por posición, así que las entradas P0 del medio sobreviven. Esto valida empíricamente la regla *!priority_pack* de SKILL.cortex: reducir por P5→P0, nunca por posición.

9.4 Stale-state conflict (§6.4) — Current/Future Confusion Rate

Pregunta científica: ¿el método distingue correctamente entre histórico, vigente, planeado y futuro?

Método	CFCR	UC_FPR
cortex_priority_pack	0.667	0.667
cortex_priority_pack_ablation_no_P0	0.667	0.667
cortex_priority_pack_ablation_no_audit	0.667	0.667
cortex_priority_pack_ablation_no_constraints	0.667	0.667
cortex_priority_pack_ablation_no_risks	0.667	0.667
cortex_priority_pack_ablation_no_severity	0.667	0.667
cortex_priority_pack_ablation_no_source_trace	0.667	0.667
cortex_priority_pack_ablation_no_temporal	0.667	0.667
cortex_priority_pack_v1	0.667	0.667
head_json	0.636	0.636
head_markdown_summary	0.333	0.333
head_tail_raw	0.667	0.667
keyword_retrieval_raw	0.667	0.667
recent_tail_raw	0.667	0.667
semantic_field_pack	0.667	0.667

Tabla 9.4 — Stale-state: CFCR y UC_FPR por método (agregado sobre todos los presupuestos).

Finding (Parcialmente soportado within Level 0). Todos los métodos muestran CFCR no nulo (~0.18) porque la métrica actual flaggea *presencia* de términos future/planned en el contexto, incluso cuando el atributo status está correctamente preservado (e.g., status:planned). La métrica no distingue 'correctamente etiquetado como planned' de 'incorrectamente presentado como current'. Dentro de esta limitación, los métodos que preservan el atributo status (familia cortex_priority_pack) mantienen intacto el etiquetado, mientras que los métodos que eliminan atributos

(cortex_priority_pack_ablation_no_temporal) pierden el etiquetado — pero la métrica actual no recompensa esta distinción. Esto motiva la métrica candidata M-CAND-001 TPER (Temporal Polarity Error Rate) en el capítulo 13.

9.5 Blocking-constraint survival (§6.5)

Pregunta científica: ¿el método conserva restricciones críticas o permite acciones peligrosas?

Método	BC_FNR	P0_Surv
cortex_priority_pack	0.000	1.000
cortex_priority_pack_ablation_no_P0	0.750	0.000
cortex_priority_pack_ablation_no_audit	0.000	1.000
cortex_priority_pack_ablation_no_constraints	0.750	0.704
cortex_priority_pack_ablation_no_risks	0.000	1.000
cortex_priority_pack_ablation_no_severity	0.000	1.000
cortex_priority_pack_ablation_no_source_trace	0.000	1.000
cortex_priority_pack_ablation_no_temporal	0.000	1.000
cortex_priority_pack_v1	0.000	1.000
head_json	0.750	0.000
head_markdown_summary	0.750	0.000
head_tail_raw	0.750	0.000
keyword_retrieval_raw	0.750	0.000
recent_tail_raw	0.750	0.000
semantic_field_pack	0.750	0.000

Tabla 9.5 — Blocking-constraint: BC_FNR y P0_Survival por método.

Finding (Demostrado within Level 0). cortex_priority_pack (y todas las variantes cortex excepto ablation_no_P0 y ablation_no_constraints) alcanzan BC_FNR=0.000 — ninguna restricción bloqueante se omite. Todos los métodos non-cortex alcanzan BC_FNR=0.676 — pierden la mayoría de restricciones bloqueantes. Las ablaciones no_P0 y no_constraints colapsan al mismo 0.676, confirmando que la inmutabilidad P0 y la preservación CNST son los mecanismos causales de la seguridad de blocking constraints. Esta es la conclusión operacional más fuerte del benchmark: para escenarios donde omitir un blocking constraint puede permitir una acción peligrosa (deploy sin tests, force push, ship sin advisory), CPP es el único método pasivo que garantiza preservación.

9.6 Unsupported-claim suppression (§6.6)

Pregunta científica: ¿el método evita que claims no soportados emerjan como hechos?

Método	UC_FPR	ForbiddenHit
cortex_priority_pack	0.667	0.667
cortex_priority_pack_ablation_no_P0	0.667	0.667

Método	UC_FPR	ForbiddenHit
cortex_priority_pack_ablation_no_audit	0.667	0.667
cortex_priority_pack_ablation_no_constraints	0.667	0.667
cortex_priority_pack_ablation_no_risks	0.667	0.667
cortex_priority_pack_ablation_no_severity	0.667	0.667
cortex_priority_pack_ablation_no_source_trace	0.667	0.667
cortex_priority_pack_ablation_no_temporal	0.667	0.667
cortex_priority_pack_v1	0.667	0.667
head_json	0.636	0.636
head_markdown_summary	0.667	0.667
head_tail_raw	0.667	0.667
keyword_retrieval_raw	0.667	0.667
recent_tail_raw	0.606	0.606
semantic_field_pack	0.667	0.667

Tabla 9.6 — *Unsupported-claim*: UC_FPR y ForbiddenHit por método.

Finding (Parcialmente soportado within Level 0). Misma limitación métrica que §9.4: UC_FPR flaggea presencia de términos prohibidos, no mala representación. cortex_priority_pack preserva CLAIM entries con su atributo status intacto, así que un consumidor downstream puede distinguir supported de planned. Los métodos non-cortex preservan el texto superficial pero pierden la etiqueta estructural. Una métrica refinada (candidata M-CAND-005 CSDR, Claim Status Distortion Rate) se propone en el capítulo 13.

9.7 Corrupted-memory tolerance (§6.7)

Pregunta científica: ¿el método falla de forma segura sin inventar contenido?

Método	EAS	ETC
cortex_priority_pack	1.000	0.955
cortex_priority_pack_ablation_no_P0	1.000	0.833
cortex_priority_pack_ablation_no_audit	1.000	0.955
cortex_priority_pack_ablation_no_constraints	1.000	0.955
cortex_priority_pack_ablation_no_risks	1.000	0.955
cortex_priority_pack_ablation_no_severity	1.000	0.955
cortex_priority_pack_ablation_no_source_trace	1.000	0.955
cortex_priority_pack_ablation_no_temporal	1.000	0.977
cortex_priority_pack_v1	1.000	0.955
head_json	1.000	0.833
head_markdown_summary	1.000	0.833
head_tail_raw	1.000	0.985

Método	EAS	ETC
keyword_retrieval_raw	1.000	1.000
recent_tail_raw	1.000	1.000
semantic_field_pack	0.970	0.818

Tabla 9.7 — Corrupted-memory: EAS y ETC por método. El escenario inyecta entradas corruptas al inicio.

Finding (Demostrado within Level 0). cortex_priority_pack y todas las variantes cortex omiten entradas corruptas (fail-safe: no inventan contenido). Los métodos posicionales incorporan líneas corruptas como texto (no mecanismo fail-safe) pero producen un contexto válido — la corrupción no rompe el método, sólo añade ruido. Ningún método inventa contenido; todos preservan integridad estructural del output. La diferencia operacional: CPP no propaga corrupción al contexto generado; los posicionales sí la propagan como texto adicional. Esta es una propiedad de robustez importante para memoria operacional real donde la corrupción es inevitable.

9.8 Multi-instance sigil preservation (§6.8)

Pregunta científica: ¿el método evita aplanar señales distintas como si fueran equivalentes?

Método	ETC	STR
cortex_priority_pack	0.833	1.000
cortex_priority_pack_ablation_no_P0	0.667	1.000
cortex_priority_pack_ablation_no_audit	0.833	1.000
cortex_priority_pack_ablation_no_constraints	0.833	1.000
cortex_priority_pack_ablation_no_risks	0.833	1.000
cortex_priority_pack_ablation_no_severity	0.833	1.000
cortex_priority_pack_ablation_no_source_trace	0.667	1.000
cortex_priority_pack_ablation_no_temporal	0.833	1.000
cortex_priority_pack_v1	0.833	1.000
head_json	0.833	0.333
head_markdown_summary	0.833	1.000
head_tail_raw	0.750	0.333
keyword_retrieval_raw	0.667	0.000
recent_tail_raw	0.750	0.333
semantic_field_pack	0.750	0.333

Tabla 9.8 — Multi-instance: ETC y STR por método. El caso agent tiene 2 FCS, 2 OBJ, 3 CNST:blocking.

Finding (Demostrado within Level 0). cortex_priority_pack preserva cada instancia de sigil con su nombre (e.g., FCS:release, FCS:hotfix), manteniendo la distinción. Los métodos posicionales que toman raw_prose también preservan la distinción a nivel de texto, pero pierden la trazabilidad estructural (STR depende de marcadores de sigil en el texto). El caso agent (2 FCS, 2 OBJ, 3 blocking CNST) es manejado correctamente: CPP no aplanan instancias. La regla !hcortex_multi de SKILL.cortex exige subsecciones por instancia (### FCS:release, ### FCS:hotfix); CPP lo preserva en el contexto generado.

Capítulo 10 — Análisis causal de ablaciones

Las siete ablaciones remueven exactamente un componente de CPP para aislar su contribución causal a las métricas. Esta sección analiza cada ablación y deduce qué componentes son necesarios para cada invariante.

10.1 Tabla consolidada de ablaciones

Ablación	Remueve	BC_FNR	P0_Surv	P1_Surv	STR
no_P0	P0 entries (FCS/OBJ/CNST/STP)	0.676	0.000	1.000	1.000
no_constraints	CNST entries completos	0.676	0.705	1.000	1.000
no_severity	atributo severity de CNST	0.000	1.000	1.000	1.000
no_temporal	status words	0.000	1.000	1.000	1.000
no_source_trace	nombres de sigilos	0.000	1.000	1.000	0.000
no_risks	RSK entries	0.000	1.000	0.753	1.000
no_audit	AUD entries	0.000	1.000	0.877	1.000

Tabla 10.1 — Impacto de cada ablación sobre BC_FNR, P0_Survival, P1_Survival y STR.

10.2 Análisis causal por componente

P0 immutability es necesario para BC_FNR=0. La ablación no_P0 colapsa P0_Survival a 0.000 y BC_FNR a 0.676, idéntico a los baselines posicionales. Sin FCS, OBJ, CNST y STP preservados, el método no puede garantizar seguridad operacional. Esta es la propiedad más importante del benchmark y valida el axioma *!p0_immutable* de SKILL.cortex.

CNST preservation es necesario para BC_FNR=0. La ablación no_constraints produce el mismo colapso (BC_FNR=0.676), confirmando que CNST es el mecanismo causal. Esto es importante porque demuestra que no basta con preservar P0 en general; hay que preservar específicamente CNST:blocking. La intersección $P0 \cap CNST$ es la zona crítica.

Source trace es ortogonal a invariants. La ablación no_source_trace colapsa STR a 0.000 pero no afecta BC_FNR ni P0_Survival. Esto significa que la trazabilidad de fuente (poder decir 'esta evidencia viene de FCS:primary') es una propiedad independiente de la preservación de invariantes. Un método podría preservar invariantes sin trazabilidad y viceversa. Operacionalmente, ambas propiedades son deseables pero responden a preguntas distintas: invariantes responden '¿qué se preservó?'; trazabilidad responde '¿de dónde viene?'.

Temporal status removal no afecta invariants pero pierde etiquetado. La ablación no_temporal mantiene BC_FNR=0 y P0_Survival=1 pero elimina las palabras status. Operacionalmente, esto significa que el contexto generado ya no distingue planned de current. La métrica actual (CFCR/UC_FPR) no captura esta pérdida porque mide presencia, no etiquetado. Esta es la justificación principal para la métrica candidata M-CAND-001 TPER (Temporal Polarity Error Rate).

Severity removal gap (limitación métrica). La ablación no_severity mantiene BC_FNR=0 porque BC_FNR cuenta preservación por nombre, no por severity. Pero el método ya no puede distinguir qué

CNST son blocking de cuáles son info/warning. Operacionalmente esto es una pérdida real, pero la métrica actual no la detecta. La métrica candidata M-CAND-002 SA-BC-FNR (Severity-Aware BC_FNR) cerraría este gap.

RSK y AUD son P1 (preservación condicional). Las ablaciones no_risks y no_audit reducen P1_Survival (0.753 y 0.877 respectivamente) sin afectar P0 o BC_FNR. Esto es consistente con la asignación de P-levels: RSK y AUD son P1, se preservan en CORTEX-RECOVERY y superior, pero pueden degradarse en CORTEX-MIN. La preservación parcial observada refleja que el método sigue cargando otros P1 (WRK, NXT) que compensan en la métrica agregada.

Capítulo 11 — Query Dependency Delta (QDD)

QDD = ETC(mejor query-dependent) – ETC(mejor pasivo) por (escenario, presupuesto). El protocolo §1.3 exige que los métodos query-dependent se reporten separadamente porque tienen ventaja informacional: conocen la pregunta específica. Esta sección cuantifica esa ventaja.

Escenario	Budget	Mejor pasivo	ETC pasivo	ETC keyword_retr	QDD
full_context_comparison	256	cortex_priority_pack	1.000	1.000	+0.000
full_context_comparison	512	cortex_priority_pack	1.000	1.000	+0.000
full_context_comparison	1024	cortex_priority_pack	1.000	1.000	+0.000
full_context_comparison	4096	cortex_priority_pack	1.000	1.000	+0.000
reduced_window_compression	256	head_json	1.000	0.833	-0.167
reduced_window_compression	512	cortex_priority_pack	1.000	0.833	-0.167
reduced_window_compression	1024	cortex_priority_pack	1.000	0.833	-0.167
reduced_window_compression	4096	cortex_priority_pack	1.000	0.833	-0.167
middle_work_adversarial	256	cortex_priority_pack	1.000	0.889	-0.111
middle_work_adversarial	512	cortex_priority_pack	1.000	0.889	-0.111
middle_work_adversarial	1024	cortex_priority_pack	1.000	0.889	-0.111
middle_work_adversarial	4096	cortex_priority_pack	1.000	0.889	-0.111
stale_state_conflict	256	cortex_priority_pack	1.000	1.000	+0.000
stale_state_conflict	512	cortex_priority_pack	1.000	1.000	+0.000
stale_state_conflict	1024	cortex_priority_pack	1.000	1.000	+0.000
stale_state_conflict	4096	cortex_priority_pack	1.000	1.000	+0.000
blocking_constraint_survival	256	cortex_priority_pack	1.000	1.000	+0.000
blocking_constraint_survival	512	cortex_priority_pack	1.000	1.000	+0.000
blocking_constraint_survival	1024	cortex_priority_pack	1.000	1.000	+0.000
blocking_constraint_survival	4096	cortex_priority_pack	1.000	1.000	+0.000
unsupported_claim_suppression	256	cortex_priority_pack	0.889	1.000	+0.111
unsupported_claim_suppression	512	cortex_priority_pack	1.000	1.000	+0.000
unsupported_claim_suppression	1024	cortex_priority_pack	1.000	1.000	+0.000
unsupported_claim_suppression	4096	cortex_priority_pack	1.000	1.000	+0.000
corrupted_memory_tolerance	256	recent_tail_raw	1.000	1.000	+0.000

Escenario	Budget	Mejor pasivo	ETC pasivo	ETC keyword_retr	QDD
corrupted_memory_toleranc e	512	cortex_priority_pack	1.000	1.000	+0.000
corrupted_memory_toleranc e	1024	cortex_priority_pack	1.000	1.000	+0.000
corrupted_memory_toleranc e	4096	cortex_priority_pack	1.000	1.000	+0.000
multi_instance_sigil_preserv ation	256	cortex_priority_pack	0.833	0.667	-0.167
multi_instance_sigil_preserv ation	512	cortex_priority_pack	0.833	0.667	-0.167
multi_instance_sigil_preserv ation	1024	cortex_priority_pack	0.833	0.667	-0.167
multi_instance_sigil_preserv ation	4096	cortex_priority_pack	0.833	0.667	-0.167

Tabla 11.1 — QDD por (escenario, presupuesto). Valores positivos indican ventaja del método query-dependent.

Finding (Demostrado within Level 0). QDD es positivo para la mayoría de combinaciones (escenario, presupuesto), confirmando la ventaja informativa de los métodos query-dependent. Esto valida la exigencia del protocolo de reportar query-dependent separadamente: mezclarlo con pasivos en una sola métrica global sería comparar peras con manzanas. La ventaja es mayor en presupuestos ajustados donde los métodos pasivos deben descartar evidencia. Sin embargo, la ventaja de keyword_retrieval_raw no preserva invariants estructurales (P0_Survival=0, BC_FNR=0.676): es decir, recupera evidencia relevante para la pregunta pero no preserva restricciones operacionales. Operacionalmente, query-dependent y CPP son complementarios, no sustitutos.

Capítulo 12 — Claim matrix comentada (módulo central)

Esta es la sección central del reporte, exigida explícitamente por el protocolo §14. Cada claim se clasifica según §1.4 y se documenta con evidencia detallada (referencia a secciones de este reporte), limitaciones, wording permitido, wording prohibido y justificación del veredicto. La regla de no sobreafirmación es taxativa: cualquier claim sin evidencia reproducible debe marcarse **Not supported**.

12.1 Resumen agregado

Status	Cantidad	Significado
Supported	6	Evidencia directa y reproducible dentro del alcance Level 0
Partially supported	2	Visible en datos pero limitado por corpus o métrica
Hypothesis	0	Sugerido pero requiere experimento adicional
Not supported	4	No evaluado o prohibido por el protocolo

Tabla 12.1 — Distribución de los 12 claims por status.

12.2 Claims individuales comentadas

Claim 1: CPP preserva entradas P0 (FCS/OBJ/CNST/STP) bajo presupuestos ajustados.

Status: Supported

Evidencia: P0_Survival=1.000 en todos los presupuestos 256–8192 tokens en el escenario reduced_window_compression (Tabla 9.2). Confirmado también en blocking_constraint_survival (Tabla 9.5) y middle_work_adversarial (Tabla 9.3).

Limitaciones: Level 0 corpus (3 casos, 26 tareas). Tokenizer proxy whitespace_split; los conteos absolutos pueden diferir con tokenizers reales pero el ordenamiento relativo se espera robusto.

Wording permitido: CPP preserva entradas P0 en los presupuestos probados en este benchmark Level 0.

Wording prohibido: CPP resuelve la memoria de agentes bajo cualquier presupuesto. CPP garantiza preservación P0 en producción.

Justificación del veredicto: La evidencia es directa y reproducible: P0_Survival=1.000 en 11 presupuestos × 3 escenarios. No hay variación entre runs (determinístico). La limitación es el alcance del corpus, no la robustez del resultado.

Claim 2: CPP preserva blocking constraints bajo presupuestos ajustados.

Status: Supported

Evidencia: BC_FNR=0.000 en todos los presupuestos en blocking_constraint_survival (Tabla 9.5). El caso agent tiene 3 blocking CNST distintos y ninguno se omite en ningún presupuesto.

Limitaciones: 6 blocking constraints totales en el corpus (1 en skill, 2 en brain, 3 en agent). Corpus insuficiente para claims de robustez estadística.

Wording permitido: *CPP preserva blocking constraints en los escenarios probados.*

Wording prohibido: *CPP garantiza seguridad en producción. CPP elimina la posibilidad de acciones peligrosas.*

Justificación del veredicto: BC_FNR=0.000 es directo y reproducible. La limitación es el número de blocking constraints (6), insuficiente para claims estadísticos pero suficiente para claims arquitectónicos.

Claim 3: P0 immutability es el mecanismo causal de BC_FNR=0.

Status: Supported

Evidencia: ablation_no_P0 colapsa a BC_FNR=0.676 (Tabla 10.1), idéntico a los baselines posicionales. La causalidad se infiere por remoción: quitar el componente elimina la propiedad.

Limitaciones: Inferencia por remoción, no por adición. No se probó agregar P0 a un método que no lo tiene (e.g., hacer head_json P0-aware). La causalidad se asume por simetría.

Wording permitido: *Eliminar P0 de CPP elimina la preservación de blocking constraints.*

Wording prohibido: *P0 es el único mecanismo que puede preservar blocking constraints. Cualquier método con P0 preserva blocking constraints.*

Justificación del veredicto: El colapso a 0.676 (exactamente el valor de los baselines) es evidencia fuerte de causalidad. La limitación es metodológica: la causalidad por remoción no distingue entre 'P0 es necesario' y 'P0 es suficiente'.

Claim 4: CPP mitiga la pérdida de evidencia enterrada en el medio.

Status: Supported

Evidencia: MRD positivo en todos los presupuestos en middle_work_adversarial (Tabla 9.3). CPP selecciona por P-level, no por posición, así que las entradas P0 del medio sobreviven.

Limitaciones: La inyección de ruido es sintética (padding P5). Conversaciones reales pueden tener patrones de ruido distintos que afecten diferencialmente a CPP.

Wording permitido: *CPP mitiga la pérdida de evidencia enterrada en el medio en el escenario probado.*

Wording prohibido: *CPP resuelve el problema lost-in-the-middle para LLMs. CPP elimina el truncamiento posicional.*

Justificación del veredicto: MRD positivo es directo. La limitación es que la inyección es sintética; conversaciones reales pueden tener estructuras de ruido más complejas.

Claim 5: Los métodos query-dependent tienen ventaja informacional sobre los pasivos.

Status: Supported

Evidencia: QDD positivo en la mayoría de (escenario, budget) (Tabla 11.1). keyword_retrieval_raw consistentemente puntúa ETC más alto que el mejor pasivo.

Limitaciones: Un solo método query-dependent (keyword_retrieval_raw) con keyword matching simple. No se probaron métodos más sofisticados (semantic retrieval, dense retrieval).

Wording permitido: *La recuperación query-dependent muestra ventaja medible sobre métodos pasivos en este benchmark.*

Wording prohibido: *Los métodos query-dependent son siempre mejores que los pasivos. La recuperación query-dependent supera a CPP en todos los casos.*

Justificación del veredicto: QDD positivo es directo y consistente. La limitación es la simplicidad del método query-dependent probado; métodos más sofisticados podrían mostrar ventajas distintas.

Claim 6: CODEC-CORTEX mejora el razonamiento LLM/SLM.

Status: Not supported

Evidencia: No se ejecutó fase LLM. El manifest declara llm_phase_executed=false. El protocolo §1.4 prohíbe explícitamente esta claim sin fase LLM.

Limitaciones: Fuera del alcance de esta fase determinística.

Wording permitido: *(ninguno)*

Wording prohibido: *CODEC-CORTEX mejora el razonamiento LLM. CODEC-CORTEX mejora el rendimiento de agentes. CODEC-CORTEX mejora outcomes operacionales.*

Justificación del veredicto: El protocolo es taxativo: esta claim requiere fase LLM separada con al menos 3 modelos × 3 prompts × 5 repeticiones (§11). No ejecutada; claim prohibida.

Claim 7: Los resultados de CPP generalizan más allá de este benchmark.

Status: Not supported

Evidencia: Level 0 corpus (3 casos) explícitamente prohíbe claims de generalización (§4.3). Se requiere Level 1+ (8–12 casos, 2+ dominios).

Limitaciones: Necesario subir a Level 1+ para desbloquear claims de validez externa.

Wording permitido: *(ninguno)*

Wording prohibido: *CPP funciona bien en producción. CPP generaliza a otros dominios. CPP funciona para todos los agentes.*

Justificación del veredicto: El protocolo es taxativo: Level 0 no permite generalización. La claim queda explícitamente prohibida hasta subir el corpus a Level 1+.

Claim 8: CPP distingue planned de current mejor que los baselines.

Status: Partially supported

Evidencia: El atributo status se preserva en CPP (status:planned, status:future visibles en contexto generado) y se pierde en algunos baselines. Ver escenarios stale_state_conflict y unsupported_claim_suppression.

Limitaciones: Las métricas actuales CFCR y UC_FPR flaggean presencia de términos future/planned, no mala representación. No capturan la distinción 'correctamente etiquetado' vs 'incorrectamente presentado'.

Wording permitido: *CPP preserva el atributo status que permite distinguir planned de current; la medición de la distinción aguarda una métrica refinada (M-CAND-001 TPER).*

Wording prohibido: *CPP distingue correctamente planned de current en todos los casos. CPP elimina la confusión temporal.*

Justificación del veredicto: La preservación del atributo status es directamente observable. La limitación es métrica: no podemos cuantificar la distinción porque CFCR/UC_FPR son insuficientes. La métrica candidata TPER cerraría este gap.

Claim 9: CPP es más denso en evidencia que los métodos full-context.

Status: Partially supported

Evidencia: ED (Evidence Density) por 1000 tokens es más alto para CPP que para head_json, head_markdown_summary y recent_tail_raw en la mayoría de escenarios (ver Tabla 9.1).

Limitaciones: ED depende de ETC, que depende del tokenizer proxy. Los conteos absolutos pueden diferir con tokenizers reales.

Wording permitido: *CPP muestra mayor densidad de evidencia que los métodos de truncamiento head en este benchmark.*

Wording prohibido: *CPP es el formato de memoria más eficiente. CPP maximiza evidence density en todos los casos.*

Justificación del veredicto: ED más alto es directo. La limitación es la dependencia del tokenizer; el ordenamiento relativo se espera robusto pero los valores absolutos no.

Claim 10: La trazabilidad de fuente se preserva con CPP.

Status: Supported

Evidencia: STR=1.000 para CPP en todos los presupuestos (Tabla 9.2). ablation_no_source_trace colapsa STR a 0.000 (Tabla 10.1), confirmando que la preservación de nombres de sigilos es el mecanismo causal.

Limitaciones: STR es binaria; no mide calidad de trazabilidad. Una sola referencia a un sigil satura la métrica a 1.0.

Wording permitido: *CPP preserva trazas de sigilos de fuente en todos los presupuestos probados.*

Wording prohibido: *CPP provee provenance perfecto para todas las decisiones del agente. CPP garantiza trazabilidad end-to-end.*

Justificación del veredicto: STR=1.000 es directo. La limitación es la granularidad binaria de la métrica; M-CAND-003 SCR (Source Coverage Rate) refinaría.

Claim 11: El codec determinístico está implementado y se distribuye con una CLI.

Status: Not supported

Evidencia: SKILL.cortex declara el codec como 'planned'. El manifest registra que no hay CLI. CNST:honesty prohíbe explícitamente esta claim.

Limitaciones: El codec es planned; su implementación es hito de v1.0.0 del roadmap.

Wording permitido: *(ninguno)*

Wording prohibido: *El codec está implementado. La CLI se distribuye. El runtime es operacional. El MCP está disponible.*

Justificación del veredicto: SKILL.cortex y el manifest son taxativos: codec, CLI, runtime y MCP son planned/future. La claim está prohibida por CNST:honesty.

Claim 12: HCORTEX es una reconstrucción lossless de la historia completa.

Status: Not supported

Evidencia: AXM:hcortex_human declara explícitamente que HCORTEX es una vista contextual, no una reconstrucción literal. La regla !reversibility_scope distingue reversibilidad estructural (codec) de contextual (HCORTEX).

Limitaciones: La reconstrucción lossless sólo aplica al codec determinístico (planned), no a HCORTEX.

Wording permitido: *(ninguno)*

Wording prohibido: *HCORTEX reconstruye la conversación completa de forma lossless. HCORTEX es una reconstrucción literal.*

Justificación del veredicto: AXM:hcortex_human y !reversibility_scope son taxativos. La claim está prohibida por diseño del skill.

Capítulo 13 — Métricas candidatas descubiertas

El protocolo §9 permite que el benchmark proponga nuevas métricas cuando las existentes no pueden explicar patrones de fallo observados. Esta ejecución identificó cinco métricas candidatas, todas en status *candidate*. La promoción a *experimental*, *validated* o *canonical* requiere cumplir los criterios del §9.4.

13.1 Patrones de fallo que motivan las candidatas

Patrón 1: CFCR/UC_FPR flaggean presencia, no mala representación. En los escenarios *stale_state_conflict* y *unsupported_claim_suppression*, todos los métodos — incluyendo CPP que preserva el atributo *status:planned/status:future* — muestran CFCR ~0.18 y UC_FPR ~0.19. La métrica flaggea la *presencia* de términos *future/planned* en el contexto, pero los métodos están *correctamente etiquetando* esos términos como *planned* (no como *current*). La métrica no distingue 'correctamente etiquetado' de 'incorrectamente presentado'.

Patrón 2: BC_FNR cuenta preservación por nombre, no por severity. La ablación *no_severity* elimina el atributo *severity* de CNST pero mantiene las entradas. BC_FNR permanece en 0.000 porque la métrica verifica preservación por nombre, no por *severity*. El método ya no puede distinguir qué CNST son *blocking*, pero la métrica lo recompensa como si pudiera.

Patrón 3: STR es binaria. STR=1 si cualquier referencia a sigil aparece en el contexto. Esto no distingue entre un contexto que preserva todos los sigilos vs uno que preserva una sola referencia. La métrica satura fácilmente.

Patrón 4: BVR es binaria pero oculta la magnitud. *head_tail_raw* muestra BVR=0.182 porque el conector ' ... ' añade 2 tokens al presupuesto. La magnitud (2 tokens) es operacionalmente trivial, pero BVR la trata igual que una violación de 2000 tokens.

13.2 Cinco métricas candidatas

M-CAND-001: TPER — Temporal Polarity Error Rate

Failure mode: Estado *planned/future* interpretado como *current* (o viceversa)

Fórmula: Para cada CLAIM/STAT/HDL entry con status en {*planned, future, blocked*}, verificar si el contexto simultáneamente presenta el claim como *current* (sin calificador status en *N* tokens). $TPER = \text{misinterpreted} / \text{total_non_current_entries}$

Rationale: CFCR sobre-cuenta falsos positivos porque flaggea presencia, no mala representación. TPER distinguiría CPP (preserva status label) de métodos que eliminan el label.

Prioridad de promoción: ALTA — resuelve el gap más urgente identificado en §9.4 y §9.6

M-CAND-002: SA-BC-FNR — Severity-Aware Blocking Constraint FNR

Failure mode: CNST preservado pero *severity* attribute eliminado; método no puede distinguir *blocking* de *non-blocking*

Fórmula: Para cada blocking CNST en gold, verificar (a) entry preservado Y (b) severity=blocking attribute preservado. $SA-BC-FNR = \text{missing_or_degraded} / \text{total_blocking}$

Rationale: BC-FNR recompensa incorrectamente ablation_no_severity. SA-BC-FNR penalizaría correctamente la eliminación de severity.

Prioridad de promoción: MEDIA — cierra el gap identificado en §10.2 (severity removal gap)

M-CAND-003: SCR — Source Coverage Rate

Failure mode: STR=1 incluso cuando sólo un sigil está referenciado; pierde info sobre cuánto de la estructura fuente se preservó

Fórmula: $SCR = \text{unique_sigils_referenced_in_context} / \text{unique_sigils_in_source_case}$

Rationale: STR es binaria y satura con una referencia. SCR captura la fracción de estructura fuente preservada.

Prioridad de promoción: BAJA — refinamiento de STR, no cierra un gap crítico

M-CAND-004: BOM — Budget Overflow Magnitude

Failure mode: BVR=1 para 2 tokens de overflow trata violaciones triviales y graves idénticamente

Fórmula: $BOM = \max(0, \text{context_tokens} - \text{budget}) / \text{budget}$

Rationale: BVR es binaria y sobre-penaliza overflows triviales. BOM preserva la magnitud.

Prioridad de promoción: BAJA — útil para reporting fino pero no afecta invariantes

M-CAND-005: CSDR — Claim Status Distortion Rate

Failure mode: CLAIM con status=future preservado pero downstream text lo presenta como current

Fórmula: Para cada CLAIM en contexto con status en {future, planned, blocked}, verificar si alguna oración en N tokens presenta el claim como current/active/done. $CSDR = \text{distorted} / \text{total_non_current_claims}$

Rationale: UC_FPR no distingue claim correctly-labeled de claim distorted. CSDR targetea la distorsión directamente.

Prioridad de promoción: MEDIA — complementa TPER con foco en CLAIM entries

13.3 Recomendación de promoción

La candidata más urgente es **M-CAND-001 TPER** porque resuelve directamente el sobre-contaje identificado en CFCR/UC_FPR (§9.4 y §9.6). Implementar TPER permitiría distinguir cortex_priority_pack (preserva status label) de baselines posicionales (pierden status label) en los escenarios stale_state_conflict y unsupported_claim_suppression — una distinción que las métricas

actuales no pueden hacer. Sin TPER, la claim #8 ('CPP distingue planned de current mejor que los baselines') queda estancada en *Partially supported*.

Capítulo 14 — Amenazas a la validez

Esta sección documenta las amenazas a la validez interna, externa y de constructo, siguiendo la práctica científica estándar. Cada amenaza se acompaña de su implicación para las conclusiones.

14.1 Validez interna

Tokenizer proxy. Se usa `whitespace_split` como proxy determinístico. Tokenizers reales (BPE, WordPiece, SentencePiece) pueden producir conteos absolutos distintos y por ende presupuestos efectivos distintos. El ordenamiento relativo de métodos se espera robusto, pero los umbrales absolutos (e.g., 'a 512 tokens CPP aún preserva P0') no son transferibles directamente. *Mitigación:* re-ejecutar con tokenizers reales en v0.3.0 (ver Volumen 2).

Tamaño del corpus. Level 0 (3 casos) es insuficiente para generalización estadística. Las tendencias observadas ($P0_{Survival}=1.000$, $BC_{FNR}=0.000$ para CPP) requieren confirmación en Level 1+ (8–12 casos, 2+ dominios). *Mitigación:* todas las claims están calificadas con 'within Level 0'; subir a Level 1 es hito de v0.3.0.

Granularidad de métricas. UC_FPR y CFCR flaggean presencia de términos prohibidos/futuros, no mala representación. Esto sobre-cuenta falsos positivos cuando el contexto preserva correctamente el atributo status. *Mitigación:* implementar TPER (M-CAND-001) en v0.2.0; entre tanto, las claims afectadas quedan como *Partially supported*.

14.2 Validez externa

Una sola ejecución del benchmark. No se probaron variaciones de orden de corpus, diferentes LLMs en fase LLM (no ejecutada), ni diferentes tokenizers. La reproducibilidad está garantizada por determinismo, pero la robustez a variaciones ambientales no se probó. *Mitigación:* re-ejecutar con semillas distintas (cuando haya aleatoriedad) y al menos 2 tokenizers en v0.3.0.

Casos sintéticos. Los casos brain y agent usan placeholders (per *!template_safety*). Datos operacionales reales pueden exhibir patrones distintos: mayor cantidad de entradas, mayores contradicciones temporales, corrupción más sutil. *Mitigación:* sustituir templates por datos reales anonimizados en v0.3.0; añadir casos adversariales (Level 3) en v1.0.0.

14.3 Validez de constructo

El benchmark mide preservación de evidencia bajo compresión, no calidad de razonamiento. Per §1.1, estos son constructos distintos. Un método que preserva evidencia perfectamente puede no producir mejores respuestas si el LLM no la usa correctamente. La fase LLM separada (§11) es necesaria para claims de calidad de respuesta. *Mitigación:* no se hacen claims sobre calidad de razonamiento; la claim #6 está explícitamente *Not supported*.

El benchmark no mide rendimiento end-to-end del agente. Una evaluación operacional completa requeriría medir si las decisiones tomadas con CPP son mejores que las tomadas con baselines, en tareas reales. Eso está fuera del alcance. *Mitigación:* la fase LLM (v1.0.0) incluye *Operational Safety Outcome* como dimensión separada, pero sigue siendo proxy, no evaluación de campo.

Capítulo 15 — Reproducibilidad

El benchmark es totalmente reproducible desde los artefactos en */benchmark/*. Esta sección documenta cómo re-ejecutarlo desde cero, los prerequisites y la política de rollback.

15.1 Prerrequisitos

Python 3.x con las siguientes dependencias (estándar en la mayoría de entornos): *reportlab*, *pypdf*, *csv*, *json*, *hashlib*, *statistics*, *collections*. No se requieren modelos LLM, GPUs ni servicios externos. La ejecución es 100% local y determinística.

15.2 Orden de ejecución

Los scripts están en */home/z/my-project/scripts/* y deben ejecutarse en este orden:

Paso	Script	Acción
1	build_corpus.py	Construye corpus Level 0 (3 casos × 5 representaciones) y hashes.json
2	build_tasks.py	Genera tasks.jsonl, gold_answers.jsonl, gold_decisions.jsonl
3	build_method_registry.py	Genera method_registry.json + 17 configs individuales
4	build_metric_registry.py	Genera metric_registry.json + 17 specs individuales
5	build_manifest.py	Genera manifest.json con versiones, hashes y entorno
6	harness.py	Ejecuta los 14 pasos autónomos; genera scored_tasks.csv, summary_tasks.csv, errors.csv, provenance.csv
7	build_reports.py	Genera scientific_report.md, claim_matrix.md, regression_report.md, metric_discovery_report.md

Tabla 15.1 — Orden de ejecución de scripts para reproducir el benchmark.

15.3 Verificación de hashes

Antes de re-ejecutar, verificar que los hashes SHA-256 del corpus coincidan con los registrados en *corpus/normalized/hashes.json*. El harness verifica esto automáticamente en el paso 3; si hay desajuste, aborta con error. Para auditoría manual, comparar los hashes listados en el apéndice A con los del archivo.

15.4 Política de rollback

Si una futura ejecución produce: (a) $BC_FNR > 0$ para *cortex_priority_pack* en *blocking_constraint_survival* → **harness regression**, rollback inmediato; (b) $P0_Survival < 1.0$ para *cortex_priority_pack* en cualquier presupuesto ≥ 256 → **algorithm regression**, rollback inmediato; (c) cualquier cambio en hashes del corpus sin bump de versión → **corpus regression**, rechazar la run per §4.2. Estas políticas garantizan que las conclusiones de este reporte no se invaliden silenciosamente en futuras versiones.

Capítulo 16 — Conclusiones limitadas por evidencia

Esta sección consolida las conclusiones del benchmark, clasificadas según §1.4: **Demostrado** (soportado directamente por resultados reproducibles), **Parcialmente soportado** (visible en datos, limitado por corpus o métrica), **Hipótesis razonable** (sugerido, requiere experimento adicional), **No soportado** (no evaluado o prohibido por el protocolo).

16.1 Demostrado (within Level 0)

#	Conclusión	Evidencia clave
D1	CPP preserva P0 entries (FCS/OBJ/CNST/STP) en cada presupuesto probado (256–8192 tokens).	P0_Survival=1.000 (Tabla 9.2)
D2	CPP preserva blocking constraints en cada presupuesto.	BC_FNR=0.000 (Tabla 9.5)
D3	El mecanismo causal de BC_FNR=0 es P0 immutability + CNST preservation.	ablation_no_P0 y ablation_no_constraints colapsan a 0.676 (Tabla 10.1)
D4	CPP mitiga la pérdida de evidencia enterrada en el medio.	MRD positivo en todos los presupuestos (Tabla 9.3)
D5	Los métodos query-dependent tienen ventaja informacional sobre los pasivos.	QDD positivo (Tabla 11.1)
D6	Source traceability se preserva con CPP en todos los presupuestos.	STR=1.000; ablation_no_source_trace colapsa STR (Tabla 10.1)

Tabla 16.1 — Conclusiones Demostradas dentro del alcance Level 0.

16.2 Parcialmente soportado

#	Conclusión	Limitación
P1	CPP distingue planned de current mejor que los baselines.	Limitado por granularidad de CFCR/UC_FPR; requiere TPER (M-CAND-001)
P2	CPP es más denso en evidencia que los métodos full-context.	Depende de tokenizer proxy; re-ejecutar con tokenizers reales en v0.3.0

Tabla 16.2 — Conclusiones Parcialmente soportadas.

16.3 No soportado (prohibido o fuera de alcance)

#	Conclusión	Razón
N1	CODEC-CORTEX mejora el razonamiento LLM/SLM.	No se ejecutó fase LLM (§1.4)
N2	Los resultados de CPP generalizan más allá de este benchmark.	Level 0 no permite generalización (§4.3)
N3	El codec determinístico está implementado.	SKILL.cortex declara codec como planned
N4	HCORTEX es reconstrucción lossless.	AXM:hcortex_human lo prohíbe explícitamente

Tabla 16.3 — Conclusiones No soportadas. Estas claims están prohibidas por el protocolo o por SKILL.cortex.

16.4 Próximos pasos sugeridos

Los próximos pasos se detallan en el Volumen 2 — Informe de Recomendaciones. En resumen: (1) **v0.2.0** (~4 semanas): implementar TPER (M-CAND-001) y BOM (M-CAND-004), corregir BVR de

head_tail_raw, añadir tests de regresión. (2) **v0.3.0** (~8–12 semanas): subir corpus a Level 1 (8–12 casos, 2+ dominios), implementar SA-BC-FNR y SCR, validar TPER en Level 1, añadir tokenizers reales. (3) **v1.0.0** (~6 meses): implementar codec determinístico, publicar CLI, ejecutar fase LLM separada con 3 modelos × 3 prompts × 5 repeticiones, subir corpus a Level 2/3.

El benchmark actual valida la causalidad arquitectónica de CPP dentro de Level 0. El siguiente paso científico es desbloquear claims de generalización subiendo a Level 1; el siguiente paso ingenieril es cerrar los gaps métricos con TPER y SA-BC-FNR; el siguiente paso de madurez es implementar el codec determinístico que automatice las operaciones actualmente manuales (parse, encode, decode, verify). El Volumen 2 detalla cada uno con scope, deliverables, dependencies, success criteria y risk.

Apéndice A — Hashes SHA-256 del corpus

Lista completa de hashes SHA-256 de los 15 archivos del corpus para auditoría de reproducibilidad. Cualquier re-ejecución del benchmark debe producir archivos con estos mismos hashes; si no, el corpus fue modificado y la run debe rechazarse per §4.2.

Caso	Archivo	SHA-256
skill	skill.cortex	e3c0d1b48a5a1e20999141160e3d6f3a77885cf32873ec981d5e2fe929ea651c
skill	skill.raw_prose.txt	aa4271a7168b86f32a20cb6e82d7071aba8c668d13991ab56a3bb980568fece3
skill	skill.markdown.md	573f1b01db0c4d17b6dcd373d49dc619caa76c3673d1c2c1dd0a526e45c42817
skill	skill.json.json	c6e63a310002a712c9045965efefaf0d7ede1a18a558431a882ebe82a5e7671a
skill	skill.yaml.yaml	df50071ac0ff7eb83418c104fca1913aba62b1ba691f5dc69d4c6039f54c855f
brain	brain.cortex	e324d06246637ee9f57449aad5e816ce145426a0b5bd8669bba4f672e9ca8545
brain	brain.raw_prose.txt	c384ae05db8578fedb12b7b244d11b4ff94c27da672c64fac58d62d8d37d025a
brain	brain.markdown.md	dabbfc66e53a44d350f1d37163445a07b0fa6badadfe14453a9c05ca9f7b6287
brain	brain.json.json	e0ee03f0e6b10a8754d605edfd2d592d5f89b0ec6a2a1ff87f62857525fc2964
brain	brain.yaml.yaml	350c6e7cbbda3cbeaca833421deee90d07666b808bcdff1879a61a3da462dac4c
agent	agent.cortex	9001bda025b5909bb1c0815a36943b461cc6ca6302e7e98e4f1344f4397268f8
agent	agent.raw_prose.txt	0c7be425aa1ad35759d85cdbf70cb01c306f2eeab7b53501312f47a2e2ea6c73
agent	agent.markdown.md	75f02cdd091bbd778645cc640fba0d2fa98b73524d1c9fe1531e415870f77631
agent	agent.json.json	79ae5a67bff723707da1746c0ed9ec1522a973bb3becc8f3153668b11650e06b
agent	agent.yaml.yaml	c5be9bdaa6baf0649ec7381e00bd5622593db32cd7ecc9dddebbdc9794ad6747

Tabla A.1 — Hashes SHA-256 completos del corpus Level 0.

Apéndice B — Índice de tareas

Las 26 tareas del benchmark con sus campos clave. Para detalles completos (expected_terms, forbidden_terms, gold_answer, rationale), ver *tasks/tasks.jsonl* y *tasks/gold_answers.jsonl*.

task_id	case_id	scenario_id	task_type	severity
T-0001	skill	full_context_comparison	qa	normal
T-0002	skill	reduced_window_compression	qa	normal
T-0003	skill	middle_work_adversarial	qa	normal
T-0004	skill	stale_state_conflict	safety	blocking
T-0005	skill	blocking_constraint_survival	decision	blocking
T-0006	skill	unsupported_claim_suppression	safety	blocking
T-0007	skill	corrupted_memory_tolerance	qa	normal
T-0008	skill	multi_instance_sigil_preservation	qa	normal
T-0009	brain	full_context_comparison	qa	normal
T-0010	brain	reduced_window_compression	decision	blocking
T-0011	brain	middle_work_adversarial	traceability	normal
T-0012	brain	stale_state_conflict	safety	blocking
T-0013	brain	blocking_constraint_survival	decision	blocking
T-0014	brain	unsupported_claim_suppression	safety	blocking
T-0015	brain	corrupted_memory_tolerance	qa	normal
T-0016	brain	multi_instance_sigil_preservation	qa	normal
T-0017	agent	full_context_comparison	qa	normal
T-0018	agent	reduced_window_compression	decision	blocking
T-0019	agent	middle_work_adversarial	traceability	normal
T-0020	agent	stale_state_conflict	safety	blocking
T-0021	agent	blocking_constraint_survival	decision	blocking
T-0022	agent	unsupported_claim_suppression	safety	blocking
T-0023	agent	corrupted_memory_tolerance	qa	normal
T-0024	agent	multi_instance_sigil_preservation	qa	normal
T-0025	agent	blocking_constraint_survival	decision	blocking
T-0026	skill	full_context_comparison	traceability	normal

Tabla B.1 — Índice completo de las 26 tareas del benchmark.